

Cross-Platform Mobile App Development

Week 2 — Environment Setup & UI/UX Basics

Instructor: Nguyen Thanh Tuan, PhD

Today's agenda

1. Set up the toolchain for all 4 stacks we'll use this semester
2. Compare the two design systems you'll build against: Material Design 3 vs. Apple HIG
3. Create an empty project in each stack (live demo)
4. Environment checklist before Week 3

Part 1: Toolchain Setup

What you need installed, by stack

Tool	PWA	Capacitor	React Native	Flutter
Node.js	✓	✓	✓	—
VS Code	✓	✓	✓	✓
Android Studio	—	✓	recommended	✓
Xcode (macOS only)	—	optional	recommended	✓
Flutter SDK	—	—	—	✓
Git	✓	✓	✓	✓

Install order this week

1. **Git** — version control, needed for every stack
2. **VS Code** + extensions (ESLint, Prettier, Flutter, Dart)
3. **Node.js** (LTS) — needed for PWA, Capacitor, React Native
4. **Android Studio** — Android SDK + emulator, needed by Capacitor, React Native, Flutter
5. **Xcode** (macOS only) — for iOS builds and simulator
6. **Flutter SDK** — `flutter doctor` must show all green before Week 9

Sanity checks

```
node -v          # should print a version, LTS recommended  
git --version  
flutter doctor   # all checks should be green (or explainable)
```

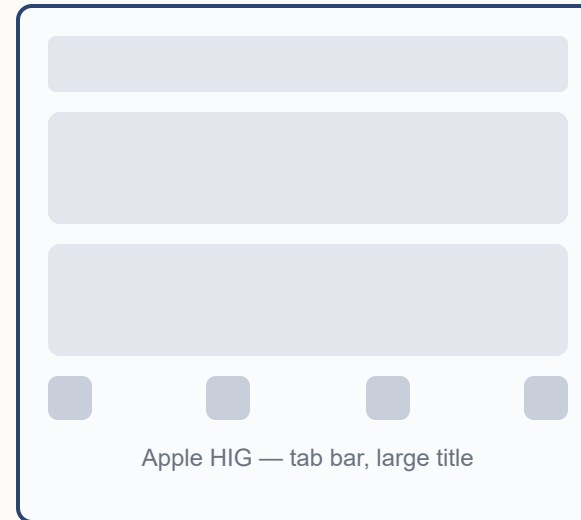
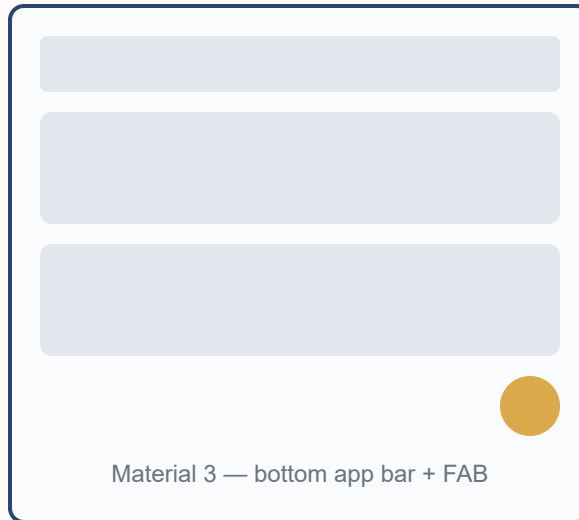
Run an Android emulator and (if on macOS) an iOS simulator at least once this week — don't wait until your first project is due to discover a broken setup.

Part 2: UI/UX Fundamentals

Two design systems you'll be building against

	Material Design 3	Apple Human Interface Guidelines (HIG)
Owner	Google	Apple
Primary platform	Android	iOS
Philosophy	Bold color, dynamic theming, elevation via shadow	Clarity, deference to content, depth via blur/layering
Navigation	Bottom nav bar, FAB	Tab bar, large titles
Typography	Roboto (default)	San Francisco (default)

The same screen, two navigation conventions



Why this matters for cross-platform apps

- Flutter ships Material by default — looks the same on iOS and Android unless you explicitly adapt it (Cupertino widgets)
- React Native uses native components by default — automatically picks up each OS's look, unless you override it
- A PWA/Capacitor app inherits whatever your web CSS defines — nothing adapts automatically

→ None of the 3 mini-projects this semester require you to hand-build both a Material and a Cupertino version — pick one design language and be consistent, but know this trade-off exists.

Part 3: Live Demo — Empty Project per Stack

Demo checklist

1. `npm create vite@latest demo-pwa -- --template vanilla` (PWA)
2. `npx cap init` on top of the PWA (Capacitor)
3. `npx create-expo-app demo-rn` (React Native via Expo)
4. `flutter create demo_flutter` (Flutter)

For each: run it, confirm it boots on an emulator/simulator/browser, and commit the empty scaffold to a new Git repo.

Environment checklist before Week 3

- [] Git configured with your name/email
- [] Node.js LTS installed, `npm -v` works
- [] VS Code with ESLint + Prettier extensions
- [] Android Studio installed, at least one emulator created and booted
- [] (macOS) Xcode installed, iOS Simulator booted at least once
- [] `flutter doctor` shows no blocking errors
- [] All 4 empty scaffolds from today's demo run successfully on your machine

Today's takeaways



You now have a working toolchain for all 4 stacks this course uses



Material Design 3 and Apple HIG are different design languages — cross-platform frameworks don't erase that difference automatically



An empty project per stack is running on your machine — no environment surprises later

Before Week 3: complete the environment checklist above.

Q&A